SHARAY

Roadmap to 10/10 — Sector-by-Sector Engineering Plan

An honest, prioritized plan for lifting every part of the platform from its current state to best-in-class, grounded in a full read of the codebase.

Author: Shardul | shardulchaulagain.com

Priorities: P0 = correctness/credibility blocker P1 = core differentiation P2 = scale/reach P3 = polish. Effort in engineer-weeks (solo).

Scorecard: current vs. target

Blended honest rating today: **6 / 10** as a deployable tool (8.5/10 as an architecture showcase). The plan below closes the gap sector by sector. The crown jewel — the verification gate — is already strong; the biggest wins are breadth, proven live paths, scale, and populated benchmarks.

Sector	Now	Target	Single biggest lever
1 Verification engine	8.0	**10**	Prove the live exploit loop end-to-end; broaden the oracle
2 Scanning & detection coverage	4.5	**10**	Real identities, more vuln classes, template depth
3 Isolation & egress sandbox	7.5	**10**	Multi-dest + IPv6 + Linux-native + seccomp/gVisor
4 Platform, scale & persistence	4.0	**10**	Parallel workers, Postgres option, real queue
5 AI reasoning layer	5.0	**10**	True graph algorithms + grounded LLM + evals
6 Reporting & evidence	7.0	**10**	Evidence bundles, SARIF/CSV, exec dashboards
7 Identity, RBAC & AppSec	6.0	**10**	MFA, retention/deletion, external notarization
8 Interactive operator console	6.5	**10**	Robust PTY, collaboration, replayable flight logs
9 Web UX & visualization	7.0	**10**	True 3D graph, live streaming, a11y, perf
10 Testing, benchmarks & QA	4.0	**10**	Populate the eval lab; regression corpus; CI gates
11 Packaging, distribution & docs	5.0	**10**	Cross-platform installers, onboarding, API docs

1. Verification engine

Now 8.0 -> Target 10 The crown jewel. Keep it the differentiator.

Where it is. 3x cold replay + negative-control canary + differential oracle + AutoPyVerifier, with the worker refusing to persist 'confirmed' unless lifecycle_state == REPRODUCED or a non-mock exit-0 proof exists. Design is excellent.

What's missing for 10/10. The live exploit-proof loop is not proven end-to-end against real targets (the AutoPyVerifier scripts reference 'body' before assignment, so they would crash on a real run). The oracle is signature/heuristic; there is no evidence-bundle sealing or chained (multi-step) verification.

Updates to make:

Concrete update	Pri	Effort
Fix the AutoPyVerifier synthesized scripts (missing body = resp.read(); add response capture) and add a test that actually runs a generated script green against the fixture.	P0	0.5w
Build a real-target regression harness: for each vuln class, a vulnerable + patched fixture that must REPRODUCE then DEMOTE. Gate CI on it.	P0	2w
Implement EXPLOITED/CHAINED/VERIFIED transitions (not just REPRODUCED): postcondition oracles and multi-step replay, sealing an evidence bundle with hashes.	P1	3w
Upgrade the differential oracle beyond fixed signatures: statistical baselining, per-target noise profiling, and tunable thresholds instead of hardcoded 35/4000ms/25%.	P1	2w
Run AutoPyVerifier scripts inside the rootless container (not a bare subprocess) so proof execution shares the egress lock.	P1	1w
Record a reproducibility score and false-positive/negative rate per assessment, surfaced in the UI and report.	P2	1w

Done = 10/10 when: Every confirmed finding ships with a sealed, re-runnable evidence bundle; the live proof loop is green in CI across a labelled corpus with measured FP/FN rates.

2. Scanning & detection coverage

Now 4.5 -> Target 10 The biggest single gap between SHARAY and Burp/Nuclei/Nessus.

Where it is. A 10-stage pipeline exists (recon, TLS, DNS, ports, crawler, DOM/SPA, schema, adaptive, differential/fuzzing/IDOR, external engines). Real coverage currently leans on Nuclei/ZAP/MSF.

What's missing for 10/10. Native detectors are early and in places synthetic: the IDOR matrix uses hardcoded 'Bearer sharay_token_alpha' identities; payload_forge/vuln_scanner cover a thin slice; no authenticated crawling; template/plugin ecosystem is absent.

Updates to make:

Concrete update	Pri	Effort
Replace synthetic auth identities with a real credential-provisioning flow (login, token capture, session reuse) so the IDOR/BOLA matrix tests actual tenants.	P0	2w
Expand native active checks to the OWASP Top 10 breadth with reproducer-backed oracles: SQLi (error/boolean/time), reflected+stored+DOM XSS, SSRF (OOB), SSTI, command injection, path traversal, open redirect, XXE, deserialization.	P1	6w
Add authenticated crawling and stateful session handling (cookies, CSRF tokens, re-login).	P1	2w
Build a signed, versioned detection-pack system (first-party templates + import of Nuclei templates) so coverage grows without core releases.	P1	3w
Add out-of-band (OOB) interaction server for blind SSRF/RCE/XXE confirmation.	P1	2w

Concrete update	Pri	Effort
Wire SBOM/dependency + secret scanners (gitleaks/semgrep/osv are present) into first-class findings with the same verification contract.	P2	2w
Protocol breadth: gRPC, WebSocket, GraphQL mutations, JWT attacks.	P2	3w

Done = 10/10 when: Coverage is measured on a public benchmark (e.g. OWASP Benchmark / Juice Shop / VAmPI) at parity with leading scanners, every detector backed by the reproducer, and detection packs ship out-of-band.

3. Isolation & egress sandbox

Now 7.5 -> Target 10 Already a strength; make it portable and provably tight.

Where it is. Fail-closed rootless containers, single-destination iptables egress allowlist with conntrack, always-teardown, shlex-quoted VM shell, /etc/hosts pinning. Honest attest().

What's missing for 10/10. Enforced egress is podman-specific, IPv4-only, and macOS/VM-centric. No per-run multi-destination or CIDR allowlist. No seccomp/AppArmor profile or userns hardening beyond cap-drop.

Updates to make:

Concrete update	Pri	Effort
Add IPv6 egress rules (ip6tables) and a multi-destination / CIDR allowlist for legitimately multi-host targets.	P0	2w
Linux-native path: enforce egress with nftables in a rootless netns without a VM, plus a Docker-rootless fallback; keep fail-closed semantics.	P1	3w
Add a seccomp profile and AppArmor/SELinux policy per tool image; consider gVisor/Kata for a stronger kernel boundary on untrusted targets.	P1	2w
Pre-build and sign the scanner images (nuclei/zap/msf/nmap/curl) and pin by digest; verify signatures before run.	P1	1.5w
Egress attestation log: record the exact ruleset applied/torn-down per run into the audit ledger.	P2	1w
Cross-platform CI that proves the allowlist actually drops non-whitelisted destinations on each OS.	P1	1.5w

Done = 10/10 when: Single-destination AND scoped multi-destination egress are enforced and CI-proven on Linux, macOS and Windows/WSL, IPv4+IPv6, with signed images and per-run egress attestation.

4. Platform, scale & persistence

Now 4.0 -> Target 10 Correct and safe, but single-box and single-worker.

Where it is. One flock-serialized worker, durable SQLite queue, bounded resources, idempotent assessments, clean lifecycle teardown.

What's missing for 10/10. No parallelism, no distributed workers, no horizontal scale; SQLite only; the queue is a polling loop; cancellation and progress are coarse.

Updates to make:

Concrete update	Pri	Effort
Introduce a proper job queue abstraction with safe concurrency (N workers, per-origin rate coordination preserved) behind the existing durable model.	P1	3w
Optional PostgreSQL backend for team/server deployments (SQLAlchemy already abstracts it); keep SQLite as the desktop default.	P1	2w

Concrete update	Pri	Effort
Distributed worker pool (serve-only nodes) with leased jobs and heartbeats for headless farms.	P2	4w
Fine-grained live progress: per-stage percentage, streamed check events over SSE/WebSocket to the UI.	P2	1.5w
Backpressure + per-org quotas so one tenant cannot starve the queue.	P2	1w
Resumable assessments and partial-result checkpoints across restarts.	P2	2w

Done = 10/10 when: A team can run many concurrent assessments across worker nodes on Postgres with live progress, per-tenant quotas, and resumable jobs — without weakening pacing or isolation.

5. AI reasoning layer

Now 5.0 -> Target 10 Great framing; make the depth match the labels.

Where it is. Deterministic-by-default attack graph, posture score, consensus/entropy escalation scaffold, 3D topology, token redaction before any provider call.

What's missing for 10/10. Min-cut/attack-path logic is heuristic rather than a proven graph algorithm; LLM output is not grounded/verified against evidence; no eval of the AI's usefulness; kill chains are partly templated.

Updates to make:

Concrete update	Pri	Effort
Replace heuristic breakpoint logic with a real max-flow/min-cut (or dominator) algorithm over the attack DAG, with unit tests on known graphs.	P1	2w
Ground LLM narratives strictly in verified findings + evidence hashes; reject/flag any claim not traceable to an observation (anti-hallucination gate).	P0	2w
Build an AI eval harness: curated cases with expected root-cause/remediation; score each provider and regression-gate changes.	P1	2w
Make remediation patches compile/apply-check against a fixture before they are shown as 'deployable'.	P1	2w
Local-model option (no external key) for fully offline reasoning parity.	P2	2w
Confidence calibration: show why each AI claim is made and its evidence provenance.	P2	1w

Done = 10/10 when: Every AI assertion is traceable to sealed evidence, graph analytics are algorithmically correct and tested, and the AI layer has a published eval score that gates regressions.

6. Reporting & evidence

Now 7.0 -> Target 10 Strong and honest; widen the formats and the proof.

Where it is. PDF/HTML/MD/JSON, vulnerability vs. posture partitioning, CWE/CVSS, protocol-accurate PoCs, redaction before storage, watermarking, async rendering with recovery.

What's missing for 10/10. No machine-ingestable standard (SARIF), no diff/trend reports across assessments, no downloadable evidence bundle, limited exec-level visualization in the PDF.

Updates to make:

Concrete update	Pri	Effort
Add SARIF and CSV exports for CI/ticketing integration; add a stable JSON schema version.	P1	1.5w

Concrete update	Pri	Effort
Attach a downloadable, hash-sealed evidence bundle (request/response captures, replay logs, AutoPyVerifier script + output) per confirmed finding.	P1	2w
Differential/trend reports: what changed vs. the previous assessment, MTTR, reopened findings.	P2	2w
Executive dashboard page in the PDF (charts: severity mix, posture trend, coverage map).	P2	1.5w
Custom branding/templating and client-ready cover + methodology/appendix sections.	P3	1w
Ticketing push (Jira/GitHub Issues) with dedupe by fingerprint.	P2	1.5w

Done = 10/10 when: Reports export to SARIF/CSV/JSON/PDF with sealed evidence bundles, trend analytics, exec dashboards and one-click ticketing — all reproducible from stored hashes.

7. Identity, RBAC & application security

Now 6.0 -> Target 10 Solid fundamentals; close the enterprise and self-security gaps.

Where it is. No default creds, >=12-char passwords, 4 roles with server-side checks, opaque sessions + CSRF, brute-force lockout, local single-use recovery, hash-linked audit ledger.

What's missing for 10/10. No MFA/SSO, local-only recovery, indefinite retention with no deletion, ledger is local tamper-evidence only, and the platform has no continuous self-security testing.

Updates to make:

Concrete update	Pri	Effort
Add MFA (TOTP/WebAuthn) and optional SSO (OIDC/SAML) for team deployments.	P1	3w
Data lifecycle: configurable retention, secure deletion/redaction of evidence and PII, and export.	P0	2w
External notarization of the audit chain (periodic signed anchor to an append-only external log) so a local admin cannot silently rewrite history.	P1	2w
Run SHARAY's own SAST/dependency/secret scanning + a security review in CI against itself.	P1	1.5w
Granular, resource-scoped permissions and per-project RBAC.	P2	2w
Signed, append-only export of the audit log for compliance.	P2	1w

Done = 10/10 when: MFA/SSO, enforceable retention + deletion, externally-anchored audit chain, and SHARAY continuously scanning itself with every finding tracked.

8. Interactive operator console

Now 6.5 -> Target 10 Distinctive feature; make it robust and collaborative.

Where it is. HITL ownership state machine, closed set of scoped verbs, target-pinned validation, egress-locked execution, exploit-gate confirmation, flight-log ingestion scaffold.

What's missing for 10/10. PTY streaming robustness and reconnection are thin; no true multi-operator collaboration; limited command surface; flight logs are not yet a replayable, auditable artifact.

Updates to make:

Concrete update	Pri	Effort
Harden PTY streaming: reconnect/resume, backpressure, binary-safe ANSI, large-output budgeting under load.	P1	2w

Concrete update	Pri	Effort
Real multi-operator sessions: presence, takeover handoff, shared read-only viewers, per-keystroke attribution in the ledger.	P2	3w
Replayable flight logs: record + deterministically replay an operator session as evidence.	P1	1.5w
Expand safe command surface (scoped) with autocomplete, history, and inline AI copilot suggestions that require human confirm.	P2	2w
Session recording export (asciinema-style) attached to the report.	P3	1w

Done = 10/10 when: Stable, reconnectable, multi-operator console with fully replayable, attributable flight logs that double as courtroom-grade evidence.

9. Web UX & visualization

Now 7.0 -> Target 10 Polished and coherent; raise it to flagship quality.

Where it is. Clean dark operator UI, honest empty states, scope preview, attack-surface topology, AI tabs, audit view, static Next.js export with CSRF-aware client.

What's missing for 10/10. Topology is a 2D force layout labelled '3D'; no live-updating assessment view; accessibility and performance on large graphs/finding sets are unverified; limited filtering/search.

Updates to make:

Concrete update	Pri	Effort
Real 3D/WebGL force graph for topology and attack paths with interactive min-cut highlighting.	P2	2w
Live assessment view: stream stage progress, checks and findings as they land (no manual refresh).	P1	1.5w
Full accessibility pass (WCAG AA): keyboard nav, ARIA, contrast, reduced-motion; add tests.	P1	1.5w
Virtualized tables + fast search/filter/sort for large finding and audit sets.	P2	1w
Saved views, bulk triage actions, and finding-status workflow UI.	P2	1.5w
Performance budget + Lighthouse CI gate.	P3	0.5w

Done = 10/10 when: A flagship, accessible, real-time UI that streams assessments live, renders true 3D graphs, and stays fast on thousands of findings.

10. Testing, benchmarks & QA

Now 4.0 -> Target 10 Good unit discipline; the headline proof is missing.

Where it is. Unit/integration/product/tool-contract suites, ruff/mypy gates, engineering/check.py reviewed gates, accessibility corpus.

What's missing for 10/10. The 'Evaluation lab' is explicitly empty (no benchmark data); no measured detection accuracy, no end-to-end live-target regression, no performance/load benchmarks, coverage unmeasured.

Updates to make:

Concrete update	Pri	Effort
Populate the evaluation lab: labelled vulnerable/patched corpora with ground truth; publish precision/recall per detector.	P0	3w
End-to-end live-target regression in CI (fixtures + intentionally-vulnerable apps) that must reproduce then remediate.	P0	2w

Concrete update	Pri	Effort
Measure and badge code coverage; add mutation testing on the verification core.	P1	1.5w
Performance/load benchmarks (throughput, latency, memory) with trend tracking.	P2	1.5w
Fuzz the parsers (headers, XML, JSONL, scope URLs) continuously.	P1	1.5w
Publish a reproducible benchmark report comparing SHARAY vs. baseline scanners.	P1	2w

Done = 10/10 when: Public, reproducible benchmarks show detection parity + near-zero false positives, the eval lab is full, and the verification core is coverage- and mutation-tested.

11. Packaging, distribution & docs

Now 5.0 -> Target 10 Runs well locally; make it effortless for anyone else.

Where it is. Desktop app (macOS .app/.command), pinned deps, migrations, preflight, synthetic fixture, now a full research paper + README.

What's missing for 10/10. macOS-centric native path, manual multi-step setup, no signed installers, no containerized one-command deploy, no hosted API docs or contributor guide.

Updates to make:

Concrete update	Pri	Effort
Cross-platform signed installers (macOS notarized, Windows, Linux AppImage/deb) with bundled runtime.	P1	3w
One-command Docker Compose / self-host deployment for the serve+worker+UI stack.	P1	1.5w
Guided first-run onboarding (owner setup, podman check, sample assessment) inside the app.	P2	1.5w
Hosted, versioned API reference (the OpenAPI route exists) + user and operator guides.	P1	1.5w
Contributor guide, detection-pack authoring docs, and a plugin/adapter SDK.	P2	2w
Telemetry-free auto-update channel for the desktop app.	P3	1.5w

Done = 10/10 when: A new user is scanning a lab in under five minutes via a signed installer or one-command deploy, with complete API/operator docs and a documented extension path.

Execution plan: four phases

Sequenced so credibility comes first (P0 fixes that unblock the headline claims), then the differentiating breadth, then scale and reach, then polish. Rough solo timeline; parallelizable with a small team.

Phase 0 - Make the claims true (approx. 6-8 weeks)

- Fix + test the AutoPyVerifier live proof loop (Sector 1).
- Real-target reproduce-then-demote regression harness in CI (1, 10).
- Replace synthetic auth identities with real credential provisioning (2).
- Anti-hallucination grounding gate on AI output (5).
- Data retention + secure deletion (7).
- Populate the evaluation lab with ground-truth corpora (10).

Phase 1 - Differentiation & breadth (approx. 3-4 months)

- OWASP Top 10 native detectors, all reproducer-backed (2).
- Detection-pack system + OOB interaction server (2).
- IPv6 + multi-destination egress; Linux-native nftables path; signed images (3).
- Evidence bundles + SARIF/CSV export (6).
- Real min-cut graph analytics + AI eval harness (5).
- Live assessment streaming + accessibility pass (9).
- MFA/SSO + externally-anchored audit chain (7).
- Published benchmark vs. baseline scanners (10).

Phase 2 - Scale & reach (approx. 2-3 months)

- Concurrent + distributed workers, Postgres backend (4).
- Multi-operator console + replayable flight logs (8).
- Trend/diff reports + ticketing integration (6).
- True 3D graph + virtualized large-data UI (9).
- Protocol breadth: gRPC/WS/GraphQL/JWT (2).

Phase 3 - Polish & distribution (approx. 1-2 months)

- Signed cross-platform installers + one-command self-host (11).
- Guided onboarding + hosted API/operator docs + plugin SDK (11).
- Custom report branding, session recordings, auto-update (6, 8, 11).

Bottom line

The hard, rare part — a principled, auditable verification engine — is already built well. Phase 0 makes its headline claims demonstrably true; Phases 1-3 add the breadth, scale and reach that separate a strong architecture from a tool teams choose over Burp and Nuclei. Executing P0+P1 alone would move the blended rating from ~6 to ~8.5; completing all four phases is what earns a genuine 10/10 everywhere.